

BÁO CÁO KỸ THUẬT

MLOPS PIPELINE

**Xây dựng hạ tầng dữ liệu và vòng đời MLOps
khép kín
cho bài toán Completion Rate Prediction
trên nền tảng AWS**

Công nghệ chính: AWS S3 · Feast Feature Store ·
MLflow
Kafka · Dagster · FastAPI · Evidently · Grafana

Dự án: **RideFlow - Predictive Ride Matching**

Biên soạn bởi: **Chu Thành Thông**

Ngày 25 tháng 6 năm 2026

Mục lục

1	Giới thiệu	1
1.1	Bối cảnh bài toán	1
1.2	Mục tiêu dự án	1
2	Kiến trúc Hệ thống Tổng quan	1
2.1	Các thành phần hạ tầng	3
2.2	Luồng dữ liệu tổng quan	4
3	Data Pipeline	4
3.1	Ingestion Pipeline	4
3.2	Feature Engineering Pipeline	4
4	Feature Store với Feast	5
4.1	Tại sao cần Feature Store?	5
4.2	Kiến trúc Feast trong hệ thống	6
4.3	Feature Views	6
5	Thử nghiệm và Huấn luyện Mô hình	7
5.1	Các mô hình được hỗ trợ	7
5.2	Quy trình huấn luyện	7
5.3	Calibration	8
5.4	Experiment Tracking với MLflow	8
5.5	Metrics đánh giá và ngưỡng chất lượng	8
6	Serving - Phục vụ Mô hình	9
6.1	Batch Prediction	9
6.2	Real-Time API	9
7	Orchestration với Dagster	10
7.1	Lịch chạy hàng ngày	10
7.2	Drift Sensor và Continuous Training	10
8	Giám sát và Drift Detection	10
8.1	Monitoring Stack	11
8.2	Metrics theo dõi	11
8.3	Vòng lặp tái huấn luyện tự động	11
9	Các vấn đề kỹ thuật đã xử lý	12
10	Tiến độ Thực hiện và Hướng Phát triển	12
10.1	Tiến độ theo giai đoạn	13
10.2	Hướng phát triển tiếp theo	13

1 Giới thiệu

1.1 Bối cảnh bài toán

Completion Rate Prediction (CRP) là bài toán dự đoán xác suất một chuyến đi được hoàn thành thành công sau khi tài xế đã nhận cuộc. Đây là một chỉ số vận hành cốt lõi trong các nền tảng gọi xe, ảnh hưởng trực tiếp đến trải nghiệm người dùng và hiệu quả phân bổ tài nguyên.

Thách thức kỹ thuật không chỉ nằm ở việc xây dựng mô hình dự đoán chính xác, mà còn ở toàn bộ vòng đời vận hành: từ thu thập dữ liệu theo thời gian thực, quản lý đặc trưng nhất quán giữa huấn luyện và phục vụ, cho đến tự động phát hiện suy giảm hiệu năng và tái huấn luyện. Dữ liệu đầu vào gồm 500.000 bản ghi lịch sử cuộc xe với nhãn nhị phân `is_completed`.

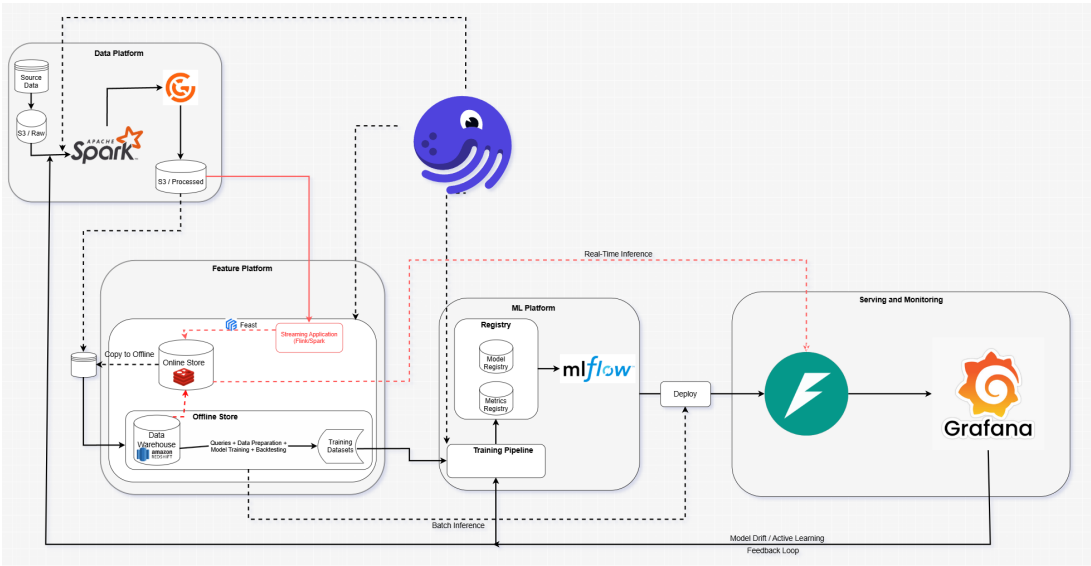
1.2 Mục tiêu dự án

Dự án tập trung vào ba trụ cột chính:

- Hạ tầng dữ liệu (Data Infrastructure):** Thiết kế pipeline nạp dữ liệu hàng ngày (Batch) và luồng sự kiện (Stream) trên AWS, đảm bảo dữ liệu sẵn sàng cho cả huấn luyện lịch sử lẫn phục vụ thời gian thực.
- Quản lý đặc trưng (Feature Management):** Triển khai Feast Feature Store để chuẩn hóa, tái sử dụng và đảm bảo tính nhất quán của các đặc trưng giữa môi trường training và serving.
- Vòng đời MLOps khép kín (Closed-loop MLOps):** Tự động hóa toàn bộ quy trình từ thử nghiệm, đánh giá, đóng gói, triển khai, giám sát đến tái huấn luyện mô hình.

2 Kiến trúc Hệ thống Tổng quan

Kiến trúc được tổ chức theo bốn lớp chức năng, từ dữ liệu thô đến kết quả dự đoán và giám sát liên tục.



Hình 1: Kiến trúc MLOps Lifecycle – RideFlow

Bảng 1: Mô tả các công cụ mã nguồn mở trong kiến trúc

Công cụ	Phân vùng	Vai trò cốt lõi
Pandas / Spark	Data Platform	Xử lý và làm sạch dữ liệu batch
Great Expectations	Data Platform	Xác thực và kiểm định chất lượng dữ liệu
Dagster	Toàn hệ thống	Điều phối luồng công việc - lập lịch, retry, sensor
Feast	Feature Platform	Kho quản lý đặc trưng - offline (Redshift) và online (Redis)
MLflow	ML Platform	Theo dõi thí nghiệm, quản lý siêu tham số và Model Registry
FastAPI	Serving	Đóng gói mô hình thành REST API phục vụ real-time inference
Evidently	Monitoring	Phát hiện data drift và concept drift
Grafana	Monitoring	Dashboard giám sát hiệu năng mô hình và hạ tầng

2.1 Các thành phần hạ tầng

Thành phần	Công nghệ	Vai trò
Data Lake	AWS S3 (Parquet)	Lưu trữ raw / processed / features / predictions
Offline Store	Amazon Redshift	Phục vụ training dataset point-in-time correct
Online Store	ElastiCache Redis	Feature serving real-time độ trễ thấp
Experiment Tracking	MLflow v3.1	Lưu vết thí nghiệm, model registry
Model Serving	FastAPI + Uvicorn	REST API inference, port 8000
Orchestration	Dagster	Lập lịch hàng ngày, sensor drift, retry policy
Streaming	Apache Kafka	Nạp sự kiện booking/driver theo thời gian thực
Monitoring	Prometheus + Grafana	Metrics, alerting, dashboard
Container	Docker Compose	Đóng gói toàn bộ stack: 9 services

Bảng 2: Tổng hợp các thành phần hạ tầng

2.2 Luồng dữ liệu tổng quan

Data Flow

Ingestion → Feature Engineering → Training / Serving

1. **Batch Ingest (01:00 UTC):** Đọc raw Parquet từ S3, làm sạch, ghi ra processed/{date}/.
2. **Feature Job (02:00 UTC):** Tính 27+ features từ processed data, ghi ra features/{date}/.
3. **Feast Materialize:** Đẩy features vào Redis để phục vụ real-time inference.
4. **Kafka Stream:** Nạp sự kiện booking/driver theo thời gian thực.
5. **Offline Store (Redshift):** Phục vụ training với dữ liệu lịch sử đầy đủ.
6. **Batch Predict (03:00 UTC):** Model đọc features, xuất predictions ra predictions/{date}/.

3 Data Pipeline

3.1 Ingestion Pipeline

Pipeline ingestion được kích hoạt mỗi ngày lúc 01:00 UTC, xử lý dữ liệu D-1:

- Đọc raw Parquet từ `s3://rideflow/raw/rides/{date}/data.parquet`
- Bỏ cột thừa, dedup theo `order_id`, chuẩn hóa kiểu ngày
- Xử lý outlier: `user_waiting_time_seconds` âm được thay bằng median
- Ghi ra `s3://rideflow/processed/{date}/data.parquet`
- Retry tự động: `max_retries=2`, `delay=30s`

3.2 Feature Engineering Pipeline

Pipeline feature đọc processed data và tạo 27+ đặc trưng qua hai module chính:

preprocessing.py – Tiền xử lý:

- Loại bỏ các cột gây *data leakage*: `est_time_arrival`, `estimate_dropoff_time`, `total_pay`
- Loại bỏ cột ID (giữ `driver_id` để tính aggregation)

- Fix giá trị âm của waiting time

transformations.py – Feature engineering:

Nhóm	Features
Supply/Demand	supply_demand_ratio, demand_supply_ratio
Confidence	eta_confidence, eda_confidence
Trip Value	fee_per_km, eta_per_km, eta_eda_ratio, pickup_to_trip_ratio
Binary Flags	is_short_trip, is_long_eta, is_high_wait, is_single_driver
Interactions	short_trip_rush, low_supply_short_trip, high_eta_rush
Temporal	hour_sin, hour_cos, minutes_since_midnight, is_weekend, is_friday
Driver Agg	driver_completion_rate_smoothed (Bayesian), driver_order_count

Bảng 3: Phân loại các nhóm đặc trưng

Lưu ý kỹ thuật: Driver completion rate sử dụng Bayesian smoothing (smoothing factor = 30) để tránh overfitting trên tài xế ít cuộc:

$$\text{smoothed_cr} = \frac{n \cdot \bar{x} + 30 \cdot \mu_{\text{global}}}{n + 30}$$

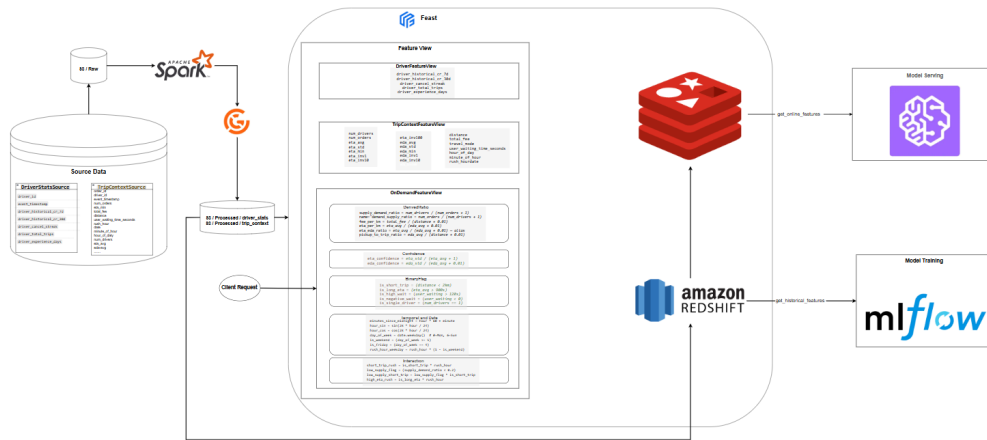
Tài xế có ít hơn 30 cuộc sẽ bị kéo về global mean, đảm bảo ước lượng đáng tin cậy.

4 Feature Store với Feast

4.1 Tại sao cần Feature Store?

Một trong những nguyên nhân phổ biến nhất gây suy giảm hiệu năng mô hình là **Training-Serving Skew** – sự không nhất quán giữa cách tính đặc trưng lúc huấn luyện và lúc phục vụ. Feature Store giải quyết vấn đề này bằng cách cung cấp *một nguồn định nghĩa duy nhất* cho tất cả features.

4.2 Kiến trúc Feast trong hệ thống



Hình 2: Kiến trúc Feast Feature Store trong RideFlow

Feast được triển khai theo mô hình dual-store:

- **Offline Store (Amazon Redshift):** Lưu trữ feature values lịch sử. Phục vụ hàm historical retrieval để tạo training dataset đúng theo thời điểm (point-in-time correct).
- **Online Store (ElastiCache Redis):** Lưu trữ feature values mới nhất với TTL 90 ngày. Phục vụ truy vấn online cho inference với độ trễ thấp.
- **Feature Registry:** Metadata trung tâm quản lý feature definitions, data sources, entity definitions.

4.3 Feature Views

Feature View	Số features	Online	Nguồn
order_raw_features	14	Có	Redshift
order_derived_features	16	Có	Redshift
on_demand_interactions	5	On-demand	Tính tại serving time

Bảng 4: Danh sách Feature Views trong Feast

On-demand Feature View tính các interaction features (hour_sin, hour_cos, short_trip_rush, ...) trực tiếp tại serving time, dùng chung logic

với training, loại bỏ hoàn toàn training-serving skew.

5 Thử nghiệm và Huấn luyện Mô hình

5.1 Các mô hình được hỗ trợ

RideFlow hỗ trợ 4 thuật toán phân loại, tất cả được cấu hình tập trung qua `models/configs/model_params.yaml` và khởi tạo qua factory pattern `build_model()`:

Mô hình	Thư viện	Hyperparameter chính
LightGBM (primary)	lightgbm	num_leaves=63, max_depth=10, n_estimators=1000, lr=0.05
XGBoost	xgboost	max_depth=8, n_estimators=1000, reg_lambda=1.0
CatBoost	catboost	depth=8, iterations=1000, l2_leaf_reg=3.0, border_count=254
Random Forest	scikit-learn	n_estimators=672, max_depth=8, max_features=0.3, class_weight=balanced

Bảng 5: Các mô hình và cấu hình hyperparameter

LightGBM được chọn làm **primary model** nhờ xây cây theo chiều lá (leaf-wise) giúp hội tụ nhanh hơn trên dữ liệu lớn, kết hợp với tốc độ training vượt trội so với các model còn lại.

5.2 Quy trình huấn luyện

Listing 1: Lệnh huấn luyện model

```
1 python -m models.training.train 2026-06-17 lgbm
2 python -m models.training.train 2026-06-17 catboost
```

Các bước trong pipeline huấn luyện (`models/training/train.py`):

- 1. Load features từ S3 (`features/{date}/features.parquet`)
- 2. Khởi tạo model qua factory với config YAML
- 3. Chạy **5-fold Stratified Cross-Validation** để đánh giá OOF

- 4. Log metrics, hyperparameters vào MLflow
- 5. Train lại trên toàn bộ dữ liệu
- 6. Log model artifact lên MLflow (artifact root: s3://rideflow/mlflow/)
- 7. Lưu 20% dữ liệu làm **reference set** cho drift detection

5.3 Calibration

Post-training calibration được áp dụng bằng **Isotonic Regression** (mặc định) hoặc Platt Scaling, đảm bảo xác suất đầu ra phản ánh đúng tần suất thực tế.

5.4 Experiment Tracking với MLflow

- **Experiment Logging:** Mỗi lần huấn luyện tự động ghi lại hyperparameters, metrics, target_date, n_samples, n_features.
- **Model Registry:** Quản lý phiên bản mô hình với trạng thái *Staging* → *Production*. Có cơ chế fallback tự động khi đăng ký thất bại.
- **Artifact Store:** Model binaries lưu trên S3 dưới định dạng skops.

5.5 Metrics đánh giá và ngưỡng chất lượng

Metric	Ý nghĩa	Ngưỡng
AUC-ROC	Khả năng phân biệt completed vs cancelled	≥ 0.80
PR-AUC	Hiệu năng trên dữ liệu mất cân bằng	-
Log Loss	Chất lượng xác suất đầu ra	≤ 0.25
F1 Score	Cân bằng precision/recall	-
ECE	Expected Calibration Error	≤ 0.05

Bảng 6: Metrics đánh giá và ngưỡng chất lượng

Model mới chỉ được promote lên Production nếu AUC-ROC không thấp hơn model Production hiện tại quá 0.01, được kiểm tra qua hàm `is_better_than_production()` so sánh với metrics lưu trong MLflow.

6 Serving - Phục vụ Mô hình

6.1 Batch Prediction

Pipeline batch prediction chạy hàng ngày lúc 03:00 UTC:

1. Load model Production từ MLflow Registry (models:/ride_completion/Production)
2. Đọc features từ S3 (24.000–500.000 rows/ngày)
3. Auto-convert cột time string dạng HH:MM sang phút
4. Chạy predict_proba, xuất completion_prob và predicted_label
5. Ghi kết quả ra s3://rideflow/predictions/{date}/predictions.parquet
6. Log prediction metrics vào MLflow: mean prob, % high prob, n_predictions

6.2 Real-Time API

Real-Time Inference Flow

order_id → **Feast Online Store (Redis)** → **On-demand Features** → **Model** → **Response**

- **Framework:** FastAPI + Uvicorn, port 8000
- **Feature retrieval:** 27 features từ 2 Feature Views qua Feast Online Store
- **On-demand:** Interaction và cyclical time features tính tại serving time
- **Latency:** Thường dưới 20ms (chủ yếu phụ thuộc Redis roundtrip)
- **Health check:** GET /health kiểm tra cả model lẫn feature store

Listing 2: Request/response real-time API

```
1 curl -X POST http://localhost:8000/predict \  
2   -H "Content-Type: application/json" \  
3   -d '{"order_id": "ORD-12345"}'  
4 # Response: {"order_id": "ORD-12345", "completion_prob": 0.8723, "  
   latency_ms": 12.3}
```

7 Orchestration với Dagster

7.1 Lịch chạy hàng ngày

Thời gian (UTC)	Pipeline	Mô tả
01:00	ingest_pipeline	Raw S3 → Processed S3
02:00	feature_pipeline	Processed → Features → Feast Materialize
03:00	inference_pipeline	Batch prediction + log metrics
On drift	retrain_pipeline	Auto-trigger khi data drift vượt ngưỡng

Bảng 7: Lịch chạy pipeline hàng ngày

Mỗi pipeline nhận `target_date` = hôm qua qua config Dagster, đảm bảo xử lý đúng partition D-1.

7.2 Drift Sensor và Continuous Training

`drift_alert_sensor` chạy tối thiểu mỗi 24 giờ, so sánh feature distribution ngày hiện tại với reference set bằng `Evidently DataDriftPreset`. Nếu `drift_share > 30%`, sensor tự động kích hoạt `retrain_pipeline` với `LightGBM`.

8 Giám sát và Drift Detection

8.1 Monitoring Stack

Công cụ	Port	Mục đích
MLflow	5000	Experiment tracking, Model Registry, artifact store
Prometheus	9090	Thu thập và lưu trữ metrics (retention: 30 ngày)
Pushgateway	9091	Tiếp nhận metrics từ batch jobs
Grafana	3000	Dashboard trực quan hoá

Bảng 8: Monitoring stack và cổng truy cập

8.2 Metrics theo dõi

Model Performance (sau D+1 khi có ground truth):

- `actual_auc_roc`, `actual_log_loss`, `actual_f1`
- `actual_completion_rate` vs `pred_completion_rate`
- `coverage` – số đơn hàng có ground truth để so sánh

Prediction Distribution (hàng ngày từ batch):

- `mean_completion_prob`, `median_completion_prob`
- `pct_high_prob` – tỷ lệ đơn có xác suất > 0.7
- `n_predictions` – tổng số dự đoán trong ngày

8.3 Vòng lặp tái huấn luyện tự động

Auto-Retraining Loop

Drift Detected → **Dagster Sensor** → **retrain_pipeline** → **MLflow Compare** → **Promote if Better**

1. Sensor phát hiện `drift_share > 30%` qua Evidently.
2. Tự động trigger `retrain_pipeline` với LightGBM trên dữ liệu mới nhất.
3. Huấn luyện xong, so sánh AUC-ROC của challenger vs production model.
4. Nếu challenger tốt hơn: đăng ký lên Staging → Production.
5. Nếu không: giữ nguyên production, log kết quả để review.

Đây là hiện thực hóa **Continuous Training (CT)** – thành phần đặc trưng của MLOps so với DevOps truyền thống.

9 Các vấn đề kỹ thuật đã xử lý

Vấn đề	Nguyên nhân	Giải pháp
Missing config “ops”	Dagster config thiếu wrapper ops	Nhập đúng cấu trúc YAML: ops.op_name.config.key
Cannot convert HH:MM	Cột time string lọt vào features	Auto-detect và convert sang minutes trong batch_predict.py
Hash mismatch Docker	PySpark install 2 lần, pip 24.0 bug	Upgrade pip trước install, pin pyspark==4.1.2
JSONDecodeError pip	Pip 24.0 bug parse large JSON PyPI	Thêm pip upgrade vào Dockerfile
GX validation lỗi	API version mismatch Great Expectations	Tạm thời bỏ qua, sẽ nâng cấp giai đoạn tiếp theo

Bảng 9: Tổng hợp các vấn đề kỹ thuật và giải pháp

10 Tiến độ Thực hiện và Hướng Phát triển

10.1 Tiến độ theo giai đoạn

Tuần	Milestone	Deliverables
1	Scope	Định nghĩa phạm vi CRP, khảo sát MLOps pipeline, thiết kế kiến trúc tổng thể.
2	Data	Xây dựng ingestion pipeline và feature engineering. Huấn luyện 4 mô hình ML, tracking qua MLflow. AUC-ROC ≥ 0.80 trên tập kiểm tra.
3	Feature Store	Triển khai Feast (Redshift offline + Redis online). Xây dựng FastAPI serving và batch prediction. Tích hợp Dagster orchestration.
4	MLOps Loop	Evidently drift detection, Prometheus + Grafana dashboard. Vòng lặp Continuous Training. Containerize stack với Docker Compose.

Bảng 10: Tiến độ thực hiện theo giai đoạn

10.2 Hướng phát triển tiếp theo

Ngắn hạn:

- Fix cột time string tại nguồn (ingest pipeline) thay vì xử lý tại inference
- Migrate khỏi MLflow Stages API (deprecated 2.9.0) sang Aliases API
- Kích hoạt lại Great Expectations sau khi nâng cấp version

Trung hạn:

- Hyperparameter Tuning tự động bằng Optuna (đã có config, chưa kích hoạt)
- Triển khai Seldon Core cho A/B testing giữa các model versions
- Tích hợp Kafka streaming pipeline cho real-time feature update

Dài hạn:

- Canary deployment với traffic splitting giữa challenger và champion
- Per-feature drift score chi tiết hơn

- Mở rộng sang multi-region với replicated Redis